Messaging: the command side, its store and one command per conversation

Where the command side's ceiling is, what the service added to it, and what one command per conversation changed. Branch `feat/messaging`, fourth session of 11 October 2026, step 8 of the tuning order in `doc/messaging-architecture.md` section 6.

1. Summary

	Before	After
The ceiling of section 7d (40 posts/s: a post took a second, acks queued to 12 s, 5 % refused)	attributed to the command path	the event store on this disk: one fsynced write at a time, 28 ms each, about 30 commands/s for every service that shares it; the same store on tmpfs takes 40 posts/s at 9 ms per post with nothing refused
Two participants posting at once	raced to the store; under load the loser retried into the same queue and collided again until its three retries were gone: 1,146 retries and 131 posts refused with 409 at 40 posts/s on disk, 3,593 retries and 842 refused at 133 posts/s on tmpfs	one command per conversation in flight on the node (`SubjectQueues`): no retry and no refusal in any burst, 17 % more posts acknowledged at 133 posts/s
Standard load, 13 posts/s	800 / 800, ack p50 52 ms	800 / 800, ack p50 53 ms
Next ceiling		the read side at 133 posts/s: relay lag 19 s, read model 31 s while the command side answers in 267 ms; step 10

2. The store

A raw probe against the store alone (HTTP, one 200-byte event per request, 1 to 40 writers) gives 25 to 34 writes per second whatever the concurrency: 28 ms per write with one writer, a queue with more (p50 900 ms with 40). Reads of a subject cost 0.4 ms, `ping` 0.2 ms, token verification 0.4 ms, so nothing per request is expensive but the write. The host's own fsync of a 4 KB file on this SSD takes 1.8 ms; one event makes the store rewrite and fsync its events segment and about twenty index files, and it commits one write at a time. A data directory bind-mounted from the SSD (the shape of the development store's volume) costs the same 28.6 ms as the container's temporary directory. The same store with `--tmpfs /data:rw,size=4g` writes 370 events per second at 2.6 ms.

Two things learned on the way: a 1 GB tmpfs makes the store switch itself to read-only after about 7,500 events (`writing is not permitted at this time`, its free-space guard), and the evaluation licence of the image caps a store at 25,000 events. Both are recorded in `doc/cqrs-reactive.md` section 7.

40 posts/s, 2,400 posts, the service unchanged	store on disk	store on tmpfs
acknowledged / refused with 409	1,282 / 131	2,398 / 0
ack latency p50 / p95 (driver)	14.6 / 31.8 s	9 / 16 ms
`messaging.command.latency {PostMessage}` p50	1.04 s	8.7 ms
`messaging.command.retries`	1,146	0

With 40 posters and a store that takes 30 writes a second, the queue in front of the store is the latency (Little's law: 40 in flight at 30 per second is 1.3 s per command). The service adds nothing of its own: on tmpfs the same 40 posts a second cost 9 ms each.

3. What the service added: collisions

Every refusal of every burst is `concurrent-modification`. Both sockets of a conversation post into the same subject; once a write waits in the store's queue, the peer's post lands in the same window, the loser re-dispatches after 5 to 50 ms, which is far less than the queue wait, and collides again until its three retries are gone. Each retry is a wasted round trip on the one resource that is the bottleneck, and the client gets a 409 for a post that did nothing wrong.

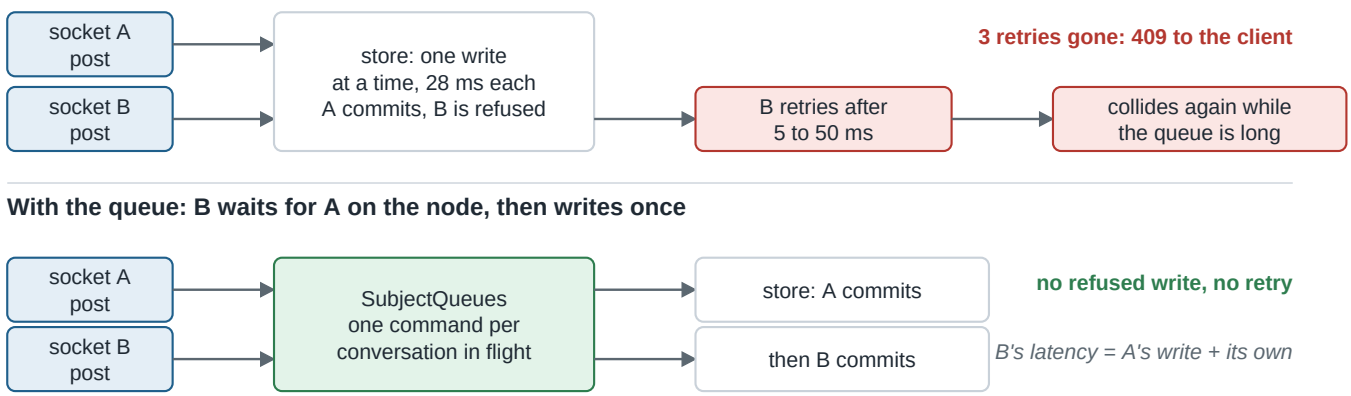

Figure 1. Two posters of one conversation on one node, before and after step 8.

`Conversations.send` now hands every dispatch to `SubjectQueues`: at most one command per subject in flight on the node; a second one waits for the first (whether it succeeds, fails or is cancelled) and then runs; commands of other subjects are not held up; the map holds only subjects with work in flight. The retry stays for commands from other nodes. `messaging.serialize-commands` turns it off, `messaging.command.queued{command}` counts the waits, and the command timer includes the wait.

4. Measurements

One node on port 8098, a fresh store for every run (on the SSD or on a 4 GB tmpfs), the development Kafka and the live MySQL, `ChatLoadDriver` with 20 conversations and 40 sockets for 60 s. Latency p50 / p95.

	without the queue	with the queue
disk, 40 posts/s: acknowledged / refused	1,282 / 131	1,448 / 0
disk, 40 posts/s: retries	1,146	0
disk, 40 posts/s: ack p50, command p50	14.6 s, 1.04 s	15.2 s, 1.74 s (the wait included)
tmpfs, 133 posts/s, 7,986 posts: acknowledged / refused	6,290 / 842	7,390 / 0
tmpfs, 133 posts/s: retries	3,593	0
tmpfs, 133 posts/s: ack p50 / p95	421 ms / 4.2 s	534 ms / 11.1 s
tmpfs, 133 posts/s: command p50 / p95	205 / 315 ms	267 / 469 ms
standard 13 posts/s, 800 posts: acknowledged, ack p50 / p95	800, 52 / 117 ms (section 7d)	800, 53 / 149 ms

No retry and no refusal in any burst, 17 % more posts acknowledged at 133 posts/s because no store round trip is wasted, and nothing changes at 13 posts/s. The command latency with the queue is higher by the wait, which is the honest number: without the queue the same post spent that time colliding and ended as a 409. On the disk store the queue cannot shorten the store's own queue (the driver's open loop keeps 40 posters sending into a store that takes 30 posts a second, so the ack p50 stays near 15 s either way); the gain there is the 131 posts that were refused and are now written. At 133 posts/s the read side of one node is the next ceiling: the relay lags 19 s and the read model 31 s while the command side answers in 267 ms. That is step 10.

5. Tests

- `SubjectQueuesTest` (new): a second command of a subject waits while another subject runs at once; a failed command releases its subject; a cancelled one does too; fifty concurrent commands of one subject never overlap; the map is empty when nothing is in flight.
- `MessagingEndToEndTest`: the racing-posts case now proves the queue instead of the retry: the two posts that race on one node are counted as queued and never as retried; the rest of the suite is unchanged. Messaging suite: 50 tests, 0 failures; db-reactive untouched by this step.
- `esdb_probe.py` (session scratchpad, not in the repository): the raw store probe; its numbers are in section 2 and in the doc.

6. Operating notes

- **The store is the budget.** On a developer disk every service that shares the store gets about 30 commands a second together. Load runs that are about a service and not the disk use a throwaway store on tmpfs (`freshstore_tmpfs.sh` in the session's scratchpad: 4 GB, not 1).
- **Production shape.** The store's write cost is the fsync pattern on the storage underneath it; an NVMe or a storage layer with a fast fsync path moves the ceiling by an order of magnitude, as tmpfs shows. Measure there before any further command-side work.
- **Cross-node races** remain possible (the two participants on different nodes posting within the same write window) and are covered by the retry as before; `messaging.command.retries` should now stay near zero on one node and small on several.
- **Back-pressure** is not added: a post that waits is better for a chat than a post refused at the door. If the queue in front of the store ever needs a bound, it belongs in front of the facade (a limit on commands in flight per node with a quick 503), not in the queue per conversation.

7. Files changed

Paths under `messaging/src/main/java/org/linlinjava/announce/messaging/` unless noted.

File	Change
application/SubjectQueues.java	new: one command per subject in flight on the node
application/Conversations.java	dispatch through the queue; messaging.command.queued
application/MessagingProperties.java	serialize-commands (default on)
infrastructure/metrics/MessagingMeters.java	COMMAND_QUEUED
test: application/SubjectQueuesTest.java	new
test: MessagingEndToEndTest.java	racing posts: queued, not retried
doc/messaging-architecture.md, doc/messaging-context.md, doc/cqrs-reactive.md	sections 2.2, 2.4, 5, 6 (step 8 done, new step 10) and the new 7g; context section 4; the store's disk behaviour in the framework doc's section 7

Branch state: uncommitted on feat/messaging, on top of this evening's statement-cost commits.